Backend Software Development

Akif Akdag (akif.akdag@uni.lu)

Lab 5 – REST



Dragon Age: The Veilguard © BioWare, EA

Due: 28 October 2026

Exercise 1 – Character Creator

In your brand new video game, players can create characters by customizing their various traits:

- id: auto-generated
- name: minimum length 2, maximum length 20
- race: human, elf, dwarf or orc
- intelligence: minimum 1, maximum 25 (default 1) [optional]

Players can create their character by sending their selection of traits as a JSON object to the endpoint **/character (POST)**. The server will auto-generate a unique 'id' for the character and store it in memory and on disk.

- If the character creation was successful, the server responds with the code 200 and returns the generated id.
- If the provided traits are invalid, e.g., race is not one of the allowed values, or the name is missing, the server responds with code 404.
- If the server runs into any internal issues, it responds with code 500.

Pay close attention to how you store the newly created character in memory and to disk!

Next, design two retrieval endpoints:

- **/characters (GET)**: this endpoint returns the full list of created characters. The response format is either in JSON or CSV, depending on the user request's header "Accept". In case of a non-supported response format, respond with the code 406.
- **/character (GET)**: if this endpoint is accessed with the parameter 'id', the server will search for a matching character and return it. If the parameter is not provided, randomly return a created character.

UUID

Want to generate a unique string? Look into the uuid module.

Validation
Want to validate a JSON object against a fixed schema? Look into the ajv module.

Express Route Parameters
Remember that route parameters in Express can rely on regular expressions (regex). Keep this in mind if, for example, you want a route parameter to be optional.

Optional: try using the async-mutex module to enable proper locking rather than solely relying on the fs module's limited capabilities when it comes to handling race conditions. Then, compare the impact on your server's performance in terms of response time using the provided load testing script under ex1-loadtest/client.js. Simply run your server in one terminal, then run the load testing client.js in a separate terminal and compare its output to your original implementation's where you did not use async-mutex. What do you notice generally?

Exercise 2 – Student ID
Design a very basic endpoint **/register (POST)** which takes as a parameter a 'name'. The server will then generate a unique hash based on the name and respond with that hash.

However, take the following into consideration:

- The unique hash generated based on the given name should always be the same, e.g., if "james" tries to register 50 times in succession, the server will respond 50 times with the same hash.
- Reflection: is your final server stateless or stateful? For example, suppose a catastrophic fire wrecks your original server and it takes 2 days to set up a new server. If "james" were to try registering again, would your server correctly respond with the same hash that was previously generated for their name? Explain through short comments in your main .js file what type of server you opted for, how it ensures consistent behavior given the previous discussion and whether performance-wise it's efficient.

Hashing
The crypto module provides useful hashing methods.